DTU

Model-based Machine Learning

Generative models

Outline

Probabilistic PCA

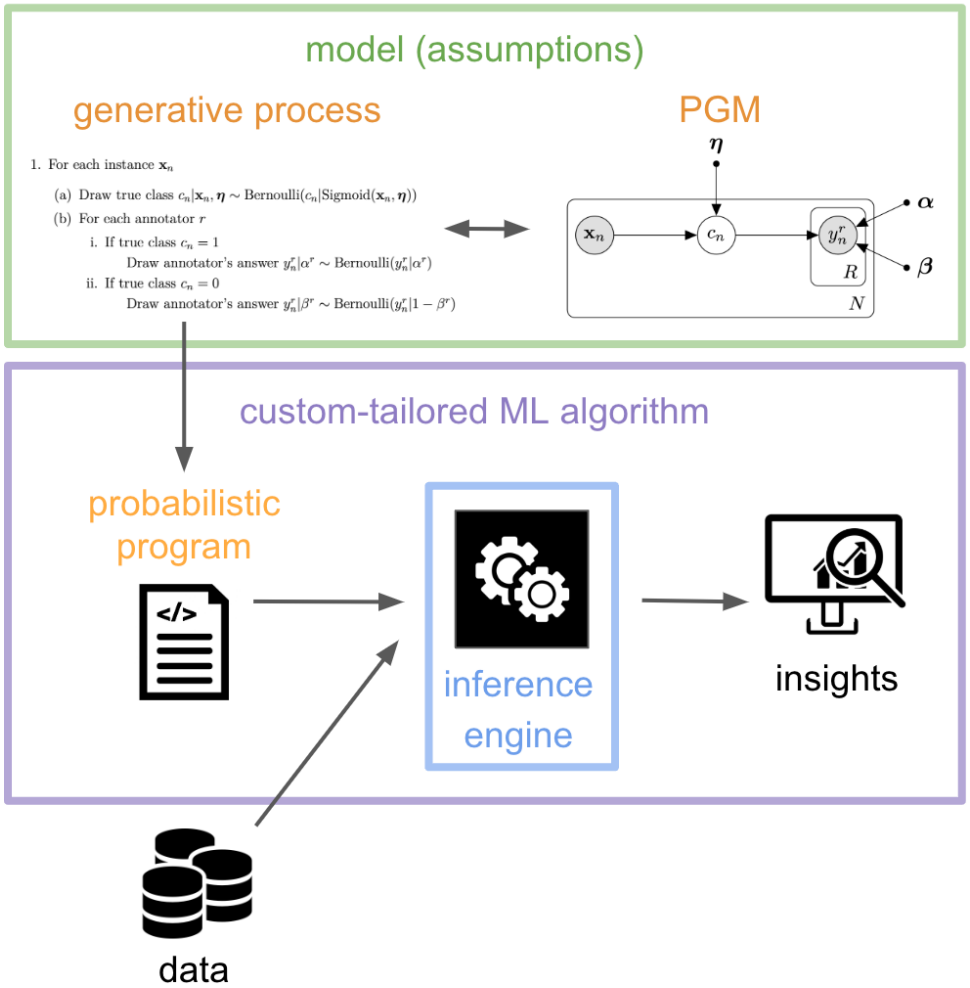
Variational Autoenconders (VAEs)

Road to MBML: where are we?

Foundations of PGMs and probabilistic programming
Probabilistic modelling in various contexts - your modelling toolbox
Bayesian inference (exact and approximate)

Week	Topic
1	Intro to the course + Prob. review
2	PGM foundations
3	PGM foundations II
4	Freq. vs Bayesian + Prob. Prog. + Mixture models
5	Regression models
6	Classification and Hierarchical models
7	Temporal models
8	Topic Models
9	Markov-chain Monte Carlo (MCMC)
10	Variational inference
11	Generative models
12	Gaussian processes
13	Project support

Model-based Machine Learning



Learning objectives

At the end of this lecture, you should be able to:

- Explain what generative models are
- Understand the relationship between PPCA and traditional PCA
- Perform Bayesian inference on PPCA
- Relate PPCA and VAEs
- Derive variational inference algorithm for VAEs
- Relate VAEs with Auto-Encoders
- Explain the encoder-decoder architecture of VAEs and how it relates to VI

Principal Component Analysis (PCA)

- Widely used dimensionality reduction technique
- Linearly project data points $\mathbf{x}_n \in \mathbb{R}^D$ onto a new coordinate system
- Directions of the new coordinate system (principal components) capture the largest variation in the data
- Principal components often computed by eigendecomposition of the data covariance matrix $\mathrm{Cov}(\mathbf{X})$ or singular value decomposition of the data matrix $\mathbf{X}$
- Retain the K principal components (eigenvectors) corresponding to the largest eigenvalues with typically $K << D$

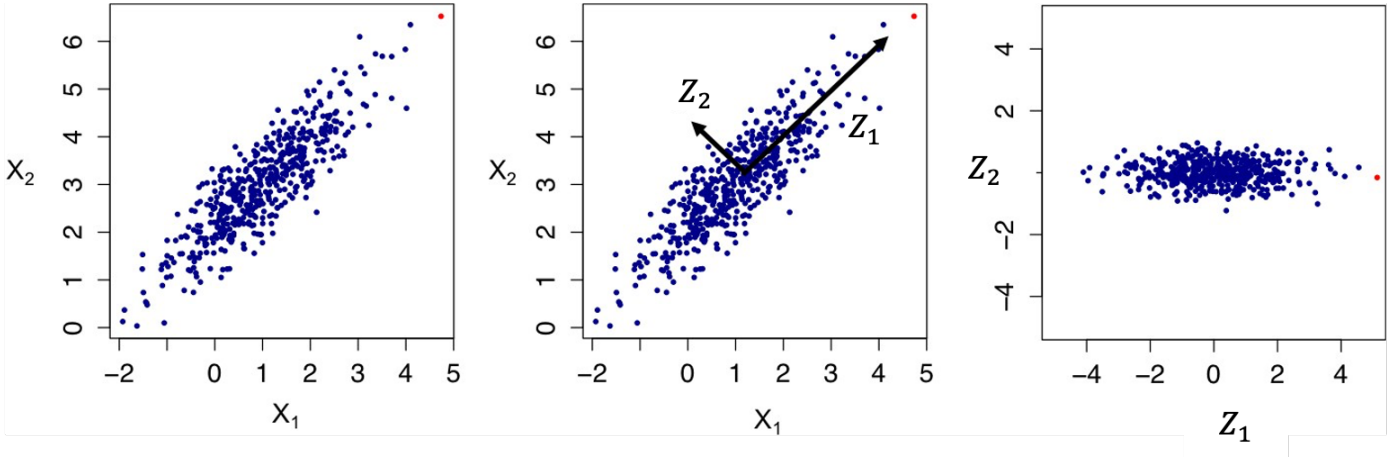


Figure: PCA example.

Probabilistic PCA

- PCA can also be expressed as the maximum likelihood solution of a probabilistic latent variable model
- Introduce an explicit latent variable $\mathbf{z}_n \in \mathbb{R}^K$ corresponding to the principal component subspace (K controls the dimensionality of the latent space)

Probabilistic PCA

- PCA can also be expressed as the maximum likelihood solution of a probabilistic latent variable model
- Introduce an explicit latent variable $\mathbf{z}_n \in \mathbb{R}^K$ corresponding to the principal component subspace (K controls the dimensionality of the latent space)
- Define a Gaussian prior distribution $p(\mathbf{z}_n)$ over the latent variable

$$p(\mathbf{z}_n) = \mathcal{N}(\mathbf{z}_n | \mathbf{0}, \mathbf{I})$$

Probabilistic PCA

- PCA can also be expressed as the maximum likelihood solution of a probabilistic latent variable model
- Introduce an explicit latent variable $\mathbf{z}_n \in \mathbb{R}^K$ corresponding to the principal component subspace (K controls the dimensionality of the latent space)
- Define a Gaussian prior distribution $p(\mathbf{z}_n)$ over the latent variable

$$p(\mathbf{z}_n) = \mathcal{N}(\mathbf{z}_n|\mathbf{0}, \mathbf{I})$$

- Define Gaussian conditional distribution $p(\mathbf{x}_n|\mathbf{z}_n)$ for the observed variable x conditioned on the value of the latent variable

$$p(\mathbf{x}_n|\ \mathbf{z}_n) = \mathcal{N}(\mathbf{x}|\mathbf{W}\mathbf{z}_n + \boldsymbol{\mu}, \sigma^2\mathbf{I})$$

- Mean of $\mathbf{x}_n$ is a linear function of $\mathbf{z}_n$ governed by the $D \times K$ matrix $\mathbf{W}$ and the D-dimensional vector $\boldsymbol{\mu}$

Probabilistic PCA

- PCA can also be expressed as the maximum likelihood solution of a probabilistic latent variable model
- Introduce an explicit latent variable $\mathbf{z}_n \in \mathbb{R}^K$ corresponding to the principal component subspace (K controls the dimensionality of the latent space)
- Define a Gaussian prior distribution $p(\mathbf{z}_n)$ over the latent variable

$$p(\mathbf{z}_n) = \mathcal{N}(\mathbf{z}_n|\mathbf{0}, \mathbf{I})$$

- Define Gaussian conditional distribution $p(\mathbf{x}_n|\mathbf{z}_n)$ for the observed variable x conditioned on the value of the latent variable

$$p(\mathbf{x}_n|\ \mathbf{z}_n) = \mathcal{N}(\mathbf{x}|\mathbf{W}\mathbf{z}_n + \boldsymbol{\mu}, \sigma^2\mathbf{I})$$

- Mean of $\mathbf{x}_n$ is a linear function of $\mathbf{z}_n$ governed by the $D \times K$ matrix $\mathbf{W}$ and the D-dimensional vector $\boldsymbol{\mu}$
- Our goal is to do inference on $\mathbf{W}$, $\boldsymbol{\mu}$, σ and $\{\mathbf{z}_n\}_{n=1}^N$

Probabilistic PCA - generative perspective

- From a generative perspective:
  1. For each observation $n \in \{1, \ldots, N\}$
     1. Sample latent projection $\mathbf{z}_n \sim \mathcal{N}(\mathbf{z}_n | \mathbf{0}, \mathbf{I})$
     2. Sample observation $\mathbf{x}_n \sim \mathcal{N}(\mathbf{x}_n | \mathbf{W}\mathbf{z}_n + \boldsymbol{\mu}, \sigma^2 \mathbf{I})$

Probabilistic PCA - generative perspective

- From a generative perspective:
  1. For each observation $n \in \{1, \ldots, N\}$
     1. Sample latent projection $\mathbf{z}_n \sim \mathcal{N}(\mathbf{z}_n | \mathbf{0}, \mathbf{I})$
     2. Sample observation $\mathbf{x}_n \sim \mathcal{N}(\mathbf{x}_n | \mathbf{W}\mathbf{z}_n + \boldsymbol{\mu}, \sigma^2 \mathbf{I})$
- Since everything is Gaussian, we can compute the marginal likelihood of the observations $\mathbf{x}_n$:

$$
\begin{aligned}
p(\mathbf{x}_n) &= \int p(\mathbf{x}_n | \mathbf{z}_n)\, p(\mathbf{z}_n)\, d\mathbf{z}_n \\
&= \int \mathcal{N}(\mathbf{x}_n | \mathbf{W}\mathbf{z}_n + \boldsymbol{\mu}, \sigma^2 \mathbf{I})\, \mathcal{N}(\mathbf{z}_n | \mathbf{0}, \mathbf{I})\, d\mathbf{z}_n \\
&= \mathcal{N}(\mathbf{x}_n | \boldsymbol{\mu}, \mathbf{W}\mathbf{W}^T + \sigma^2 \mathbf{I})
\end{aligned}
$$

  where we used the well-known results from the Exact Inference lecture (slide 7)
- Note: if we standardize the observations $\mathbf{x}_n$, we can set $\boldsymbol{\mu} \triangleq 0$!

Probabilistic PCA - generative perspective

- From a generative perspective:
  1. For each observation $n \in \{1, \ldots, N\}$
     1. Sample latent projection $\mathbf{z}_n \sim \mathcal{N}(\mathbf{z}_n|\mathbf{0}, \mathbf{I})$
     2. Sample observation $\mathbf{x}_n \sim \mathcal{N}(\mathbf{x}_n|\mathbf{W}\mathbf{z}_n + \boldsymbol{\mu}, \sigma^2\mathbf{I})$
- Since everything is Gaussian, we can compute the marginal likelihood of the observations $\mathbf{x}_n$:

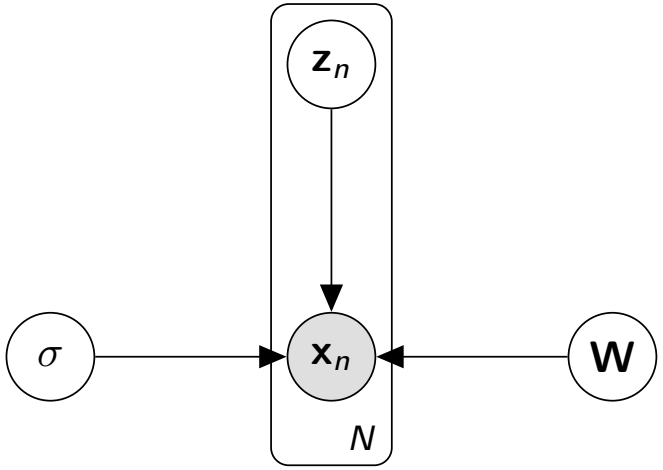
$$
\begin{aligned}
p(\mathbf{x}_n) &= \int p(\mathbf{x}_n|\mathbf{z}_n)\, p(\mathbf{z}_n)\, d\mathbf{z}_n \\
&= \int \mathcal{N}(\mathbf{x}_n|\mathbf{W}\mathbf{z}_n + \boldsymbol{\mu}, \sigma^2\mathbf{I})\, \mathcal{N}(\mathbf{z}_n|\mathbf{0}, \mathbf{I})\, d\mathbf{z}_n \\
&= \mathcal{N}(\mathbf{x}_n|\boldsymbol{\mu}, \mathbf{W}\mathbf{W}^T + \sigma^2\mathbf{I})
\end{aligned}
$$

  where we used the well-known results from the Exact Inference lecture (slide 7)
- Note: if we standardize the observations $\mathbf{x}_n$, we can set $\boldsymbol{\mu} \triangleq 0$!
- We can find maximum lik. estimates of $\mathbf{W}$ and σ (Bishop, 2006, pp. 574-577)

$$
\mathbf{W}_{\text{MLE}}, \sigma_{\text{MLE}} = \arg\max_{\mathbf{W},\sigma} \sum_{n=1}^{N} p(\mathbf{x}_n)
$$

Probabilistic PCA - fully Bayesian version

- Generative process:
  1. For each column $k \in \{1, \ldots, K\}$ in the matrix $\mathbf{W} = (\mathbf{w}_1, \ldots, \mathbf{w}_K)^T$
     1. Sample $\mathbf{w}_k \sim \mathcal{N}(\mathbf{w}_k | \, \mathbf{0}, \lambda \mathbf{I})$
  2. Sample $\sigma^2 \sim \text{InvGamma}(\sigma^2 | \, \alpha, \beta)$
  3. For each observation $n \in \{1, \ldots, N\}$
     1. Sample latent projection $\mathbf{z}_n \sim \mathcal{N}(\mathbf{z}_n | \mathbf{0}, \mathbf{I})$
     2. Sample observation $\mathbf{x}_n \sim \mathcal{N}(\mathbf{x}_n | \mathbf{W}\mathbf{z}_n, \sigma^2 \mathbf{I})$
- As a graphical model:



Probabilistic PCA - advantages

- Advantages of Probabilistic PCA over regular PCA:
  - Bayesian inference is possible, with all the advantages it entails

Probabilistic PCA - advantages

- Advantages of Probabilistic PCA over regular PCA:
  - Bayesian inference is possible, with all the advantages it entails
  - Provides a principled way of handling missing values in the data

Probabilistic PCA - advantages

- Advantages of Probabilistic PCA over regular PCA:
  - Bayesian inference is possible, with all the advantages it entails
  - Provides a principled way of handling missing values in the data
  - We can learn the dimensionality K using Automatic Relevance Determination (ARD) - see Variational Inference lecture

Probabilistic PCA - advantages

- Advantages of Probabilistic PCA over regular PCA:
  - Bayesian inference is possible, with all the advantages it entails
  - Provides a principled way of handling missing values in the data
  - We can learn the dimensionality K using Automatic Relevance Determination (ARD) - see Variational Inference lecture
  - Existence of a likelihood function allows direct comparison with other probabilistic density models

Probabilistic PCA - advantages

- Advantages of Probabilistic PCA over regular PCA:
  - Bayesian inference is possible, with all the advantages it entails
  - Provides a principled way of handling missing values in the data
  - We can learn the dimensionality K using Automatic Relevance Determination (ARD) - see Variational Inference lecture
  - Existence of a likelihood function allows direct comparison with other probabilistic density models
  - Can be easily extended to handle categorical data, count data, etc., or combinations of those - just choose $p(\mathbf{x}|\mathbf{z})$ accordingly

Probabilistic PCA - advantages

- Advantages of Probabilistic PCA over regular PCA:
  - Bayesian inference is possible, with all the advantages it entails
  - Provides a principled way of handling missing values in the data
  - We can learn the dimensionality K using Automatic Relevance Determination (ARD) - see Variational Inference lecture
  - Existence of a likelihood function allows direct comparison with other probabilistic density models
  - Can be easily extended to handle categorical data, count data, etc., or combinations of those - just choose $p(\mathbf{x}|\mathbf{z})$ accordingly
  - Once fitted, can be run generatively to provide samples of "fake data" $\mathbf{x}_*$:
    1. Sample latent projection $\mathbf{z}_* \sim \mathcal{N}(\mathbf{z}|\mathbf{0}, \mathbf{I})$
    2. Sample observation $\mathbf{x}_* \sim \mathcal{N}(\mathbf{x}|\hat{\mathbf{W}}\mathbf{z}_*, \hat{\sigma}^2\mathbf{I})$

    where $\hat{\mathbf{W}}$ and $\hat{\sigma}^2$ are either samples from their posterior distributions or point estimates found with MLE/MAP

Playtime!

- Probabilistic PCA in Pyro
  - See “11 - PPCA.ipynb" notebook
  - Expected duration: 45 minutes

From linear to non-linear projections

- In PPCA, the relationship between $\mathbf{z}$ and $\mathbf{x}$ was a linear Gaussian

$$p(\mathbf{x}_n|\mathbf{z}_n) = \mathcal{N}(\mathbf{x}_n|\mathbf{W}\mathbf{z}_n + \boldsymbol{\mu}, \sigma^2\mathbf{I})$$

- Why not replace it with a non-linear relationship?

$$p(\mathbf{x}_n|\mathbf{z}_n) = \mathcal{N}(\mathbf{x}_n|f_{\boldsymbol{\theta}}(\mathbf{z}_n), \sigma^2\mathbf{I})$$

where $f_{\boldsymbol{\theta}}$ is a non-linear function parameterized by $\boldsymbol{\theta}$

From linear to non-linear projections

- In PPCA, the relationship between $\mathbf{z}$ and $\mathbf{x}$ was a linear Gaussian

$$p(\mathbf{x}_n|\mathbf{z}_n) = \mathcal{N}(\mathbf{x}_n|\mathbf{W}\mathbf{z}_n + \boldsymbol{\mu}, \sigma^2\mathbf{I})$$

- Why not replace it with a non-linear relationship?

$$p(\mathbf{x}_n|\mathbf{z}_n) = \mathcal{N}(\mathbf{x}_n|f_{\boldsymbol{\theta}}(\mathbf{z}_n), \sigma^2\mathbf{I})$$

  where $f_{\boldsymbol{\theta}}$ is a non-linear function parameterized by $\boldsymbol{\theta}$
- What function? A neural network is a great candidate!
- Generative process becomes:
  1. For each observation $n \in \{1, \ldots, N\}$
     1. Sample latent projection $\mathbf{z}_n \sim \mathcal{N}(\mathbf{z}_n|\mathbf{0}, \mathbf{I})$
     2. Sample observation $\mathbf{x}_n \sim \mathcal{N}(\mathbf{x}_n|f_{\boldsymbol{\theta}}(\mathbf{z}_n), \sigma^2\mathbf{I})$

From linear to non-linear projections

- In PPCA, the relationship between $\mathbf{z}$ and $\mathbf{x}$ was a linear Gaussian

$$p(\mathbf{x}_n|\mathbf{z}_n) = \mathcal{N}(\mathbf{x}_n|\mathbf{W}\mathbf{z}_n + \boldsymbol{\mu}, \sigma^2\mathbf{I})$$

- Why not replace it with a non-linear relationship?

$$p(\mathbf{x}_n|\mathbf{z}_n) = \mathcal{N}(\mathbf{x}_n|f_{\boldsymbol{\theta}}(\mathbf{z}_n), \sigma^2\mathbf{I})$$

  where $f_{\boldsymbol{\theta}}$ is a non-linear function parameterized by $\boldsymbol{\theta}$
- What function? A neural network is a great candidate!
- Generative process becomes:
  1. For each observation $n \in \{1, \ldots, N\}$
     1. Sample latent projection $\mathbf{z}_n \sim \mathcal{N}(\mathbf{z}_n|\mathbf{0}, \mathbf{I})$
     2. Sample observation $\mathbf{x}_n \sim \mathcal{N}(\mathbf{x}_n|f_{\boldsymbol{\theta}}(\mathbf{z}_n), \sigma^2\mathbf{I})$
- Goal: compute posterior distributions over $\{\mathbf{z}_n\}_{n=1}^N$ and also estimate the neural network parameters $\boldsymbol{\theta}$
- How? Variational inference!

Variational inference

- Joint probability: $p(\mathbf{X}, \mathbf{Z}) = \prod_{n=1}^{N} p(\mathbf{z}_n)\, p(\mathbf{x}_n | \mathbf{z}_n)$

Variational inference

- Joint probability: $p(\mathbf{X}, \mathbf{Z}) = \prod_{n=1}^{N} p(\mathbf{z}_n)\, p(\mathbf{x}_n|\mathbf{z}_n)$
- Variational approximation:

$$q(\mathbf{Z}) = \prod_{n=1}^{N} q(\mathbf{z}_n|\boldsymbol{\nu}_n) = \prod_{n=1}^{N} \mathcal{N}(\mathbf{z}_n|\boldsymbol{\nu}_n)$$

Variational inference

- Joint probability: $p(\mathbf{X}, \mathbf{Z}) = \prod_{n=1}^{N} p(\mathbf{z}_n)\, p(\mathbf{x}_n|\mathbf{z}_n)$
- Variational approximation:

$$q(\mathbf{Z}) = \prod_{n=1}^{N} q(\mathbf{z}_n|\boldsymbol{\nu}_n) = \prod_{n=1}^{N} \mathcal{N}(\mathbf{z}_n|\boldsymbol{\nu}_n)$$

- Goal: find the variational parameters $\{\boldsymbol{\nu}_n\}$ that maximize the ELBO $\mathcal{L}(q)$

$$\mathcal{L}(q) = \sum_{n=1}^{N} \mathbb{E}_q[\log p(\mathbf{z}_n, \mathbf{x}_n)] - \mathbb{E}_q[\log q(\mathbf{z}_n|\boldsymbol{\nu}_n)]$$

Variational inference

- Joint probability: $p(\mathbf{X}, \mathbf{Z}) = \prod_{n=1}^{N} p(\mathbf{z}_n)\, p(\mathbf{x}_n|\mathbf{z}_n)$
- Variational approximation:

$$q(\mathbf{Z}) = \prod_{n=1}^{N} q(\mathbf{z}_n|\boldsymbol{\nu}_n) = \prod_{n=1}^{N} \mathcal{N}(\mathbf{z}_n|\boldsymbol{\nu}_n)$$

- Goal: find the variational parameters $\{\boldsymbol{\nu}_n\}$ that maximize the ELBO $\mathcal{L}(q)$

$$\mathcal{L}(q) = \sum_{n=1}^{N} \mathbb{E}_q[\log p(\mathbf{z}_n, \mathbf{x}_n)] - \mathbb{E}_q[\log q(\mathbf{z}_n|\boldsymbol{\nu}_n)]$$

- Problem: does not work well in practice! A much better choice of $q(\mathbf{Z})$ is:

$$q(\mathbf{Z}) = \prod_{n=1}^{N} q(\mathbf{z}_n|g_\phi(\mathbf{x}_n)) = \prod_{n=1}^{N} \mathcal{N}(\mathbf{z}_n|g_\phi(\mathbf{x}_n))$$

  where g is a neural network parameterized by ϕ
- Why? The number of variational parameters does not scale with N anymore! Also, we can extrapolate the inference process across observations $\mathbf{x}_n$.

Variational inference

- Joint probability: $p(\mathbf{X}, \mathbf{Z}) = \prod_{n=1}^{N} p(\mathbf{z}_n)\, p(\mathbf{x}_n|\mathbf{z}_n)$
- Variational approximation:

$$q(\mathbf{Z}) = \prod_{n=1}^{N} q(\mathbf{z}_n|\boldsymbol{\nu}_n) = \prod_{n=1}^{N} \mathcal{N}(\mathbf{z}_n|\boldsymbol{\nu}_n)$$

- Goal: find the variational parameters $\{\boldsymbol{\nu}_n\}$ that maximize the ELBO $\mathcal{L}(q)$

$$\mathcal{L}(q) = \sum_{n=1}^{N} \mathbb{E}_q[\log p(\mathbf{z}_n, \mathbf{x}_n)] - \mathbb{E}_q[\log q(\mathbf{z}_n|\boldsymbol{\nu}_n)]$$

- Problem: does not work well in practice! A much better choice of $q(\mathbf{Z})$ is:

$$q(\mathbf{Z}) = \prod_{n=1}^{N} q(\mathbf{z}_n|g_\phi(\mathbf{x}_n)) = \prod_{n=1}^{N} \mathcal{N}(\mathbf{z}_n|g_\phi(\mathbf{x}_n))$$

  where g is a neural network parameterized by ϕ
- Why? The number of variational parameters does not scale with N anymore! Also, we can extrapolate the inference process across observations $\mathbf{x}_n$.
- You can think of g as a neural network that learns to do inference! This is called *Amortized Variational Inference*

Variational inference procedure

Repeat until convergence:

1. Compute parameters of the variational approximation $g_\phi(\mathbf{x}_n)$ for each $\mathbf{x}_n$

Variational inference procedure

Repeat until convergence:

1. Compute parameters of the variational approximation $g_\phi(\mathbf{x}_n)$ for each $\mathbf{x}_n$
2. Take a sample from the variational dist. $\mathbf{z}_n^{(s)} \sim \mathcal{N}(\mathbf{z}_n | g_\phi(\mathbf{x}_n))$ for each $\mathbf{x}_n$

Variational inference procedure

Repeat until convergence:

1. Compute parameters of the variational approximation $g_\phi(\mathbf{x}_n)$ for each $\mathbf{x}_n$
2. Take a sample from the variational dist. $\mathbf{z}_n^{(s)} \sim \mathcal{N}(\mathbf{z}_n | g_\phi(\mathbf{x}_n))$ for each $\mathbf{x}_n$
3. Use the samples $\{\mathbf{z}_n^{(s)}\}_{n=1}^N$ to approximate expectations in the ELBO:

$$
\begin{aligned}
\mathcal{L}(q) &= \sum_{n=1}^N \mathbb{E}_q[\log p(\mathbf{z}_n, \mathbf{x}_n) - \mathbb{E}_q[\log q(\mathbf{z}_n | g_\phi(\mathbf{x}_n))] \\
&\approx \sum_{n=1}^N \log p(\mathbf{z}_n^{(s)}, \mathbf{x}_n) - \log q(\mathbf{z}_n^{(s)} | g_\phi(\mathbf{x}_n)) \\
&= \sum_{n=1}^N \log \mathcal{N}(\mathbf{x}_n | f_\theta(\mathbf{z}_n^{(s)}), \sigma^2 \mathbf{I}) + \log \mathcal{N}(\mathbf{z}_n^{(s)} | \mathbf{0}, \mathbf{I}) - \log \mathcal{N}(\mathbf{z}_n^{(s)} | g_\phi(\mathbf{x}_n))
\end{aligned}
$$

Variational inference procedure

Repeat until convergence:

1. Compute parameters of the variational approximation $g_\phi(\mathbf{x}_n)$ for each $\mathbf{x}_n$
2. Take a sample from the variational dist. $\mathbf{z}_n^{(s)} \sim \mathcal{N}(\mathbf{z}_n | g_\phi(\mathbf{x}_n))$ for each $\mathbf{x}_n$
3. Use the samples $\{\mathbf{z}_n^{(s)}\}_{n=1}^N$ to approximate expectations in the ELBO:

$$\begin{aligned}
\mathcal{L}(q) &= \sum_{n=1}^N \mathbb{E}_q[\log p(\mathbf{z}_n, \mathbf{x}_n) - \mathbb{E}_q[\log q(\mathbf{z}_n | g_\phi(\mathbf{x}_n))] \\
&\approx \sum_{n=1}^N \log p(\mathbf{z}_n^{(s)}, \mathbf{x}_n) - \log q(\mathbf{z}_n^{(s)} | g_\phi(\mathbf{x}_n)) \\
&= \sum_{n=1}^N \log \mathcal{N}(\mathbf{x}_n | f_\theta(\mathbf{z}_n^{(s)}), \sigma^2 \mathbf{I}) + \log \mathcal{N}(\mathbf{z}_n^{(s)} | \mathbf{0}, \mathbf{I}) - \log \mathcal{N}(\mathbf{z}_n^{(s)} | g_\phi(\mathbf{x}_n))
\end{aligned}$$

4. Use gradients $\nabla_\theta \mathcal{L}(q)$ and $\nabla_\phi \mathcal{L}(q)$ to update θ and ϕ respectively

Variational inference procedure

Repeat until convergence:

1. Compute parameters of the variational approximation $g_\phi(\mathbf{x}_n)$ for each $\mathbf{x}_n$
2. Take a sample from the variational dist. $\mathbf{z}_n^{(s)} \sim \mathcal{N}(\mathbf{z}_n | g_\phi(\mathbf{x}_n))$ for each $\mathbf{x}_n$
3. Use the samples $\{\mathbf{z}_n^{(s)}\}_{n=1}^N$ to approximate expectations in the ELBO:

$$\begin{aligned}
\mathcal{L}(q) &= \sum_{n=1}^N \mathbb{E}_q[\log p(\mathbf{z}_n, \mathbf{x}_n) - \mathbb{E}_q[\log q(\mathbf{z}_n | g_\phi(\mathbf{x}_n))] \\
&\approx \sum_{n=1}^N \log p(\mathbf{z}_n^{(s)}, \mathbf{x}_n) - \log q(\mathbf{z}_n^{(s)} | g_\phi(\mathbf{x}_n)) \\
&= \sum_{n=1}^N \log \mathcal{N}(\mathbf{x}_n | f_\theta(\mathbf{z}_n^{(s)}), \sigma^2 \mathbf{I}) + \log \mathcal{N}(\mathbf{z}_n^{(s)} | \mathbf{0}, \mathbf{I}) - \log \mathcal{N}(\mathbf{z}_n^{(s)} | g_\phi(\mathbf{x}_n))
\end{aligned}$$

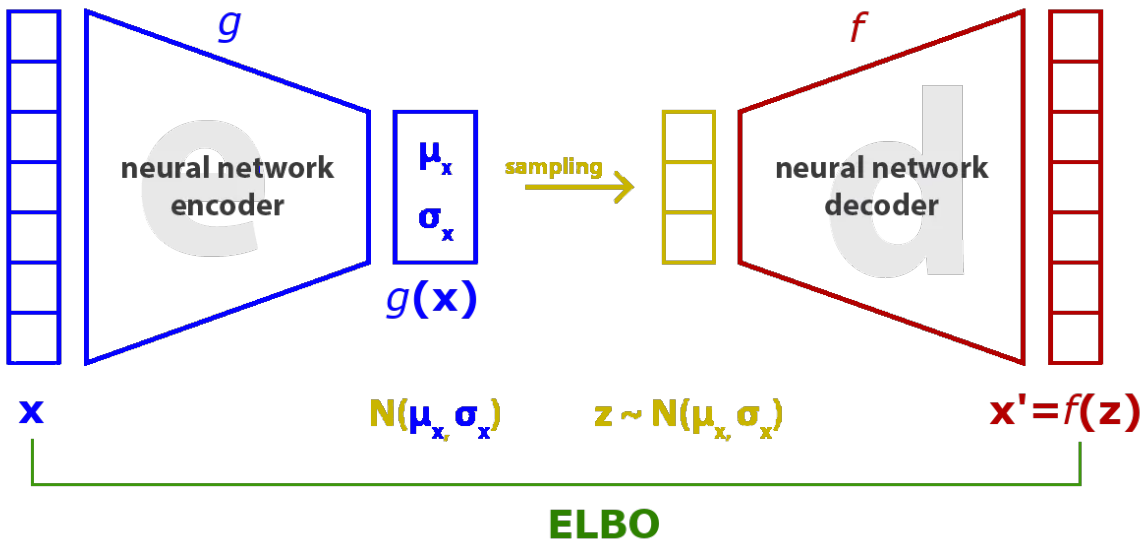
4. Use gradients $\nabla_\theta \mathcal{L}(q)$ and $\nabla_\phi \mathcal{L}(q)$ to update θ and ϕ respectively

In other words:

1. Pass observations $\mathbf{x}_n$ through g
2. Sample $\mathbf{z}_n^{(s)} \sim \mathcal{N}(\mathbf{z}_n | g_\phi(\mathbf{x}_n))$
3. Pass samples $\mathbf{z}_n^{(s)}$ through f
4. Compute ELBO and take gradient step

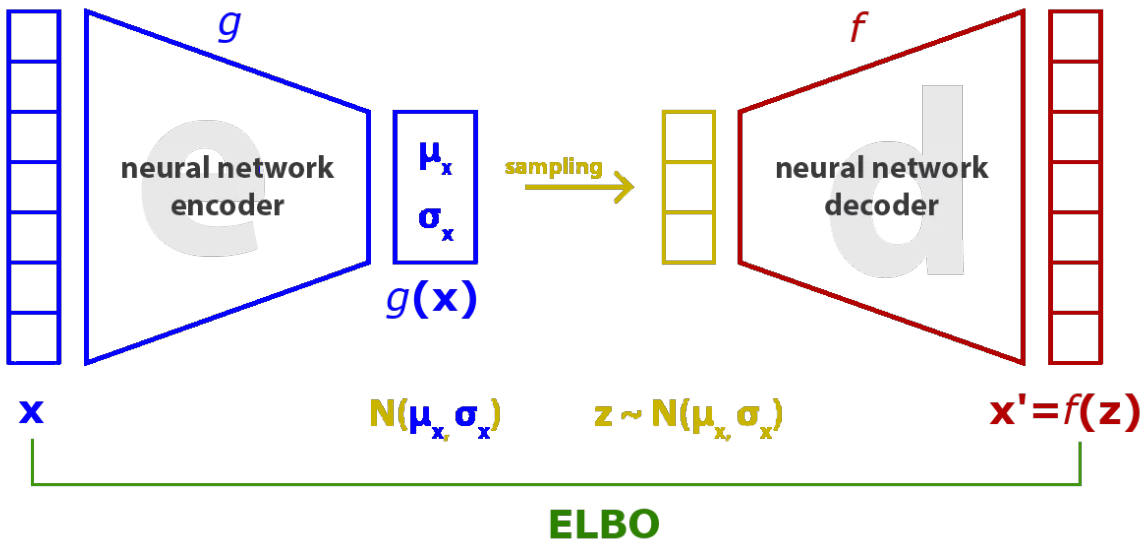
Variational Autoencoder (VAE)

1. Pass observations $\mathbf{x}_n$ through g $\rightarrow$ **the encoder network**
2. Sample $\mathbf{z}_n^{(s)} \sim \mathcal{N}(\mathbf{z}_n | g_\phi(\mathbf{x}_n))$
3. Pass samples $\mathbf{z}_n^{(s)}$ through f $\rightarrow$ **the decoder network**
4. Compute ELBO (**loss function**) and take gradient step $\rightarrow$ **backward pass**



Variational Autoencoder (VAE)

1. Pass observations $\mathbf{x}_n$ through g $\rightarrow$ **the encoder network**
2. Sample $\mathbf{z}_n^{(s)} \sim \mathcal{N}(\mathbf{z}_n | g_\phi(\mathbf{x}_n))$
3. Pass samples $\mathbf{z}_n^{(s)}$ through f $\rightarrow$ **the decoder network**
4. Compute ELBO (**loss function**) and take gradient step $\rightarrow$ **backward pass**

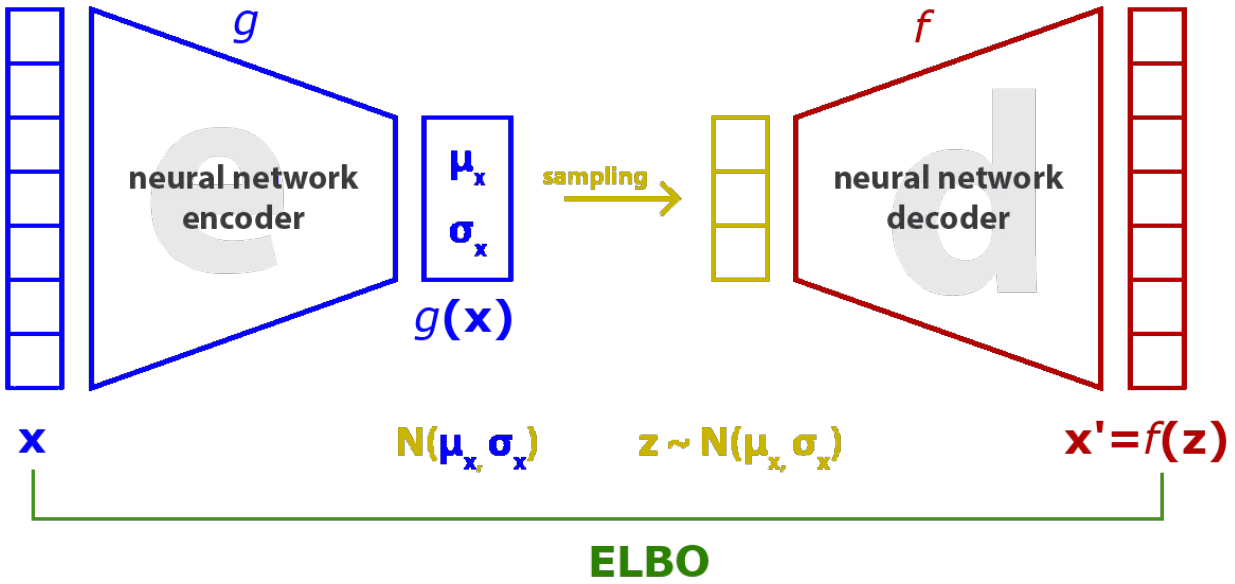


- We can train it end-to-end (as a single neural network with parameters $\{\boldsymbol{\theta}, \boldsymbol{\psi}\}$) if we define:

$$\mathbf{z}_n^{(s)} = \boldsymbol{\mu}_\mathbf{x} + \boldsymbol{\sigma}_\mathbf{x} \times \boldsymbol{\epsilon}_n, \quad \boldsymbol{\epsilon}_n \sim \mathcal{N}(\mathbf{0}, \ \mathbf{I})$$

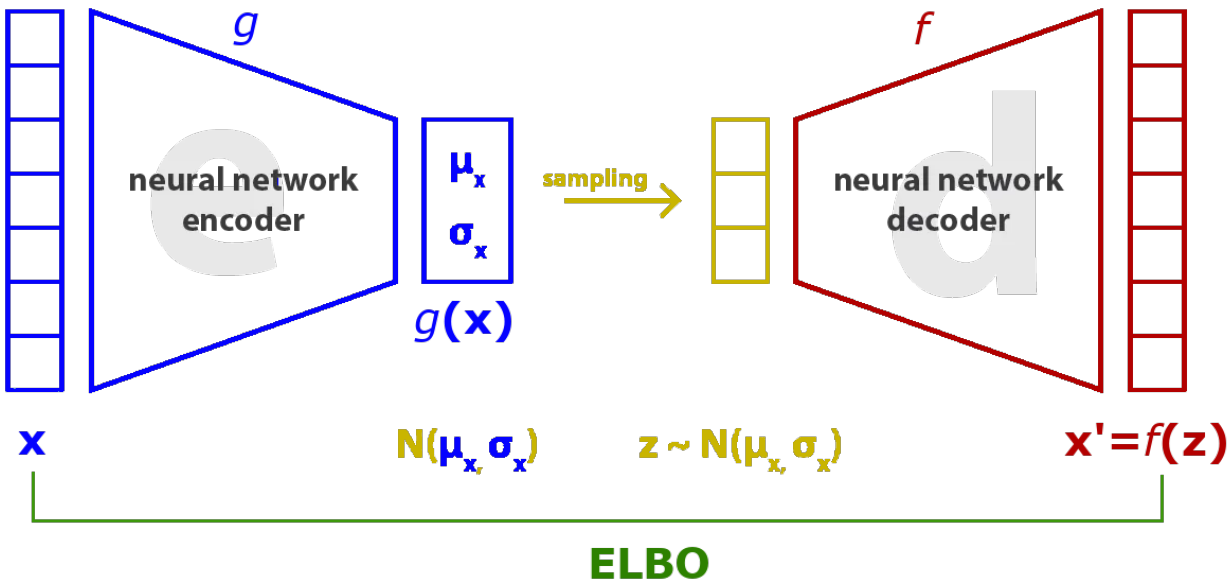
- This is know as the *reparameterization trick*

Variational Autoencoder (VAE)



- Training this neural network in this way is the same as doing variational inference on the probabilistic model described earlier (slide 10)!

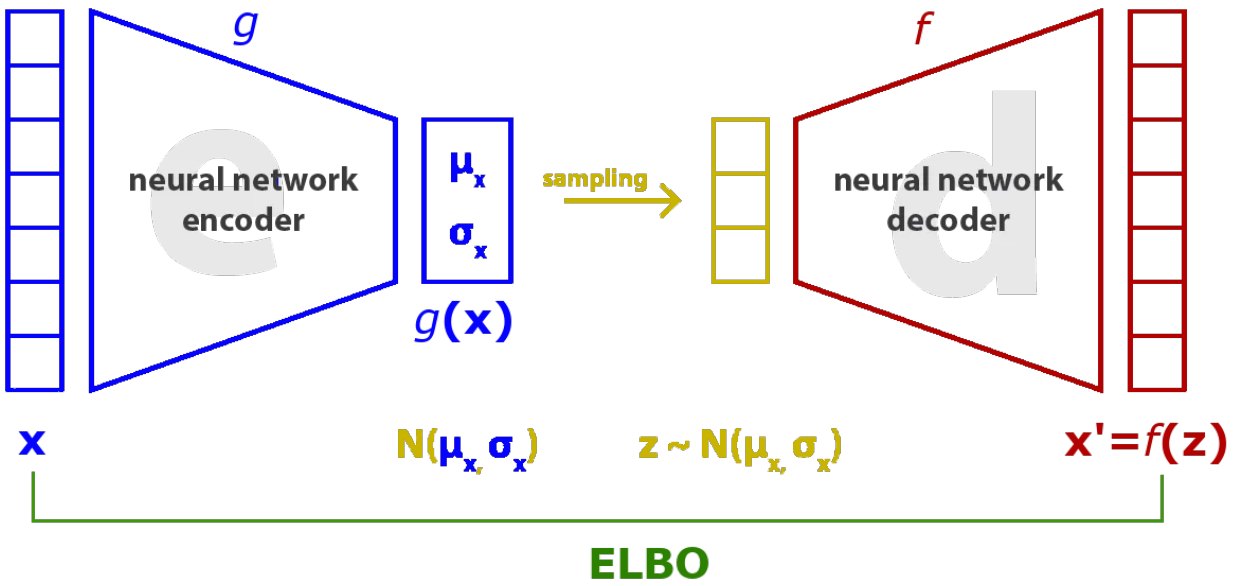
Variational Autoencoder (VAE)



- Training this neural network in this way is the same as doing variational inference on the probabilistic model described earlier (slide 10)!
- Once the model is trained, we can:
  - Project arbitrary data points $\mathbf{x}_*$ to the latent space using the encoder:

$$\mathbf{z}_* = g_\phi(\mathbf{x}_*)$$

Variational Autoencoder (VAE)



- Training this neural network in this way is the same as doing variational inference on the probabilistic model described earlier (slide 10)!
- Once the model is trained, we can:
  - Project arbitrary data points $\mathbf{x}_*$ to the latent space using the encoder:

$$\mathbf{z}_* = g_\phi(\mathbf{x}_*)$$

  - Obtain new samples $\mathbf{x}_{\text{new}}$ using the decoder ("generative" mode):

$$\mathbf{x}_{\text{new}} = f_\theta(\mathbf{z}_{\text{new}}), \quad \mathbf{z}_{\text{new}} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

Playtime!

- VAE in Pyro
  - See “11 - VAE.ipynb" notebook
  - Expected duration: 1h